



Projeto Piloto TEK

Back-End

2025/1

Por Pedro Sodré



Projeto – Rede Social da CT

1. Descrição do projeto

Neste Projeto Piloto, você irá desenvolver uma aplicação chamada Instagram da CT Junior, com funcionalidades básicas de uma rede social de compartilhamento de fotos. A ideia é construir uma aplicação funcional que inclua:

- Telas de Login e Cadastro
- Tela de Feed de Postagens
- Tela de Meu Perfil
- Tela de Postar
- Mecanismo de Logout
- Navegação via **Sidebar** para manter o controle entre as telas

Você terá acesso ao Figma do projeto como referência visual:

➡ Link do Figma: [Acessar Figma](#)

2. Entrega

Data de entrega: até XX/XX/2025 (23:59).

Envio para: tecnologia@ctjunior.com.br com o assunto "Entrega PP – Nome".

Entregáveis:



- 1) Link do repositório no GitHub.
- 2) Relatório individual com experiência, desafios e soluções. De preferência em pdf, espaçamento 1.5 e fonte Arial 12. Escreva no máximo 3 páginas e sinta-se livre para dividir os tópicos.
- 3) O relatório pode ser enviado até 24h após o projeto.

3. A se fazer

Antes de detalhar as rotas back-end, vale destacar: [O FRONT-END PRECISA DE VOCÊ](#). Caso você não forneça as rotas, o dinamismo dos dados no front-end fica comprometido. O seu repositório será cobrado pelas pessoas do front-end para que possam integrar a camada visual com a camada de dados.

Algumas tecnologias que devem ser utilizadas:

1. TypeScript.
2. Prisma ORM / Prisma Client.
3. Fastify.
4. Docker.
5. PostgreSQL – banco de dados relacional.
6. Bcrypt – Gerar senhas encriptadas.
7. Fastify Jwt.

1) Login ([POST] '/login'):

O usuário deve enviar email e senha e receber um token JWT do back-end. Esse token JWT deve ser armazenado em forma de cookie pelo front-end. O token deve ser assinado pelo id do



usuário para que ele tenha como acessar os próprios dados dentro da rede social. Devolva o status 200. O token deve ser expirado em 2h.

```
Retorno:
{
  token:
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJub21lIjoiaS2FtaWthemUiLCJzZW50YSI6Im1haW9yLWRvLWVzdGFkbyIsImNwZil6IjEwMS4wMTAuMTAxLTAxliwiY3RqdW5pb3liOiJBIDAxIGRvIGVzdGFkbywgZGVtIHF1ZSB5ZXNwZWl0YXliLCJpYXQiOiE3MTY2NTgwMDAsImV4cCI6MTcxNjY2NTIwMH0.avc1241ASFAS534uhIGYFiGDAsd7"
}
```

OBSERVAÇÃO IMPORTANTE: Todo token jwt pode ser decodificado. Como exemplo, acesse [este link](#), role para baixo e coloque o token grifado acima no campo “Encoded” para ver a decodificação do token.

Portanto, **NUNCA** assine um token JWT com e-mail, documento, senha(mesmo que encriptada), ou qualquer outro dado sensível do usuário, pois o token pode ser decodificado e as informações, vazadas. Sempre assine com o ID do usuário.

2) Criar usuário ([POST] ‘/user’):

O usuário deve se cadastrar fornecendo um e-mail válido único, uma senha e um ‘username’ único. Não autorize o cadastro de uma conta caso o username ou o e-mail já exista no banco de dados. Envie o status 409 caso algum desse conflito aconteça. Em caso de sucesso, devolva o status 201. Lembre-se de encriptar a senha do usuário com bcrypt e salvar a senha no banco de forma **encriptada**. Essa rota não deve exigir token jwt para funcionar.



3) Retornar posts ([GET] '/posts'):

O usuário deve poder ver todos os posts em ordem mais recente. Retorne um vetor de posts contendo todos os posts do sistema em ordem respeitando o seguinte critério: o post criado mais recentemente deve ser enviado nos menores índices. ex: post mais recente na posição 0, post mais antigo, na última posição. Devolva o status 200. Caso o usuário não forneça um token válido, retorne o status 401 e não permita a rota devolver os posts.

retorno deve ser algo como:

```
{
  posts: [
    {
      username: "pedro_sodrem",
      post_photo: "link.com",
      description: null,
      user_photo: "https://sodre.png",
      post_id:
        "b2d874ab-054d-4363-b920-19875800e803"
    },
    {
      username: "raphaelcdornelas",
      post_photo: "link2.com",
      description: "CT JUNIOR 01 DO ESTADO.",
    }
  ]
}
```



```
        user_photo: "https://raphael.png",
        post_id:
        "a2c274ab-024e-2318-fd20-12725800k018a"
    },
]
}
```

4) Editar perfil([PATCH] '/user')

O usuário deve poder alterar sua foto de perfil. Receba um novo link e altere a coluna do usuário. Não autorize um usuário trocar a foto de outro usuário. Caso isso ocorra ou o usuário não forneça um JWT, retorne o status 401 e não permita a atualização. Lembre-se de pegar o id do usuário pelo jwt enviado.

Sem retorno, apenas status 200 ou 204.

5) Editar post([PATCH] '/post/:id')

O usuário deve poder editar um post próprio alterando a descrição do post. Caso o usuário não tenha um token JWT ou tente alterar um post que não é dele, negue a requisição enviando o status 401. Em caso de sucesso, apenas envie o status 200.

Sem retorno, apenas o status 200 ou 204.

6) Deletar post([DELETE] '/post/:id')

O usuário deve poder deletar um post próprio. Caso o usuário não tenha um token JWT ou tente alterar um post que não é dele, negue a requisição enviando o status 401. Em caso de



post não encontrado, retorne o status 404.

Sem retorno, apenas o status 200.

7) Criar post([POST] '/post')

Crie um post recebendo descrição opcional e um link obrigatório de foto. Forneça por padrão um id aleatório(UUID), o id do usuário que está fazendo o post(recupere o id pelo token JWT) e a data que está sendo postado(coloque sempre a data do momento que receber a data – new Date()). Retorne o status 201 caso o post seja criado com sucesso. Caso o usuário não envie um token JWT válido, negue a requisição enviando o status 401.

Sem retorno, apenas o status 201.

8) Retornar meus posts ([GET] '/my-posts'):

O usuário deve poder ver todos os **próprios** posts em ordem mais recente. Devolva o status 200. Caso o usuário não tenha um token jwt válido, retorne o status 401 e não permita a rota devolver os posts. Retorno:

```
{
  user_photo: "https://enrico.png",
  username: "enricomassariol",
  posts: [
    {
      foto: "https://link.com",
```



```
        description: "VAS-CO",
        post_id:
        "b2d874ab-054d-4363-b920-19875800e803"
    },
    {
        foto: "https://link2.com",
        description: null,
        post_id:
        "4e853645-a949-42bf-8b19-120368a18d60"
    }
]
}
```

5. Finalização do Projeto

Após entregar, você fará uma apresentação individual presencial em formato de slide, podendo mostrar seu trabalho como forma de apoio. Datas e tempos de apresentação serão informados ao fim do projeto piloto.

6. Considerações e Dicas:

- 1) Deixe a validação por **JWT** para ser realizado primeiro em conjunto com a rota de login. A autenticação é fundamental para o front-end e para as demais rotas do back-end.
- 2) Não deixe imperfeições para trás. Se você puder corrigir ou melhorar algo, não faça "vista grossa".

- 3) Não use inteligência artificial, nós sabemos ler o código de vocês e entendemos completamente qual código foi gerado por IA e qual não. Como motivação: este projeto piloto não foi feito para abordar funcionalidades complexas e você possivelmente será cobrado para explicar o seu código.
- 4) Busque ajuda com membros da diretoria. Você possivelmente precisará de suporte com Prisma e GitHub – *Não tenha vergonha. Nós todos já fizemos o Projeto Piloto e participamos do Case. Estivemos no seu lugar um dia e possivelmente trabalharemos juntos.* Alguns membros que podem te ajudar de forma especializada: Enrico e Silvio.
- 5) Não se preocupe em saber como o front-end funciona. Em algum momento você saberá front-end e se tornará um full-stack. Tenha calma.
- 6) Banco de dados:

☐ User:

- username: string – username do usuário.
- email: string – email do usuário.
- senha: string – lembre de encriptar.
- fotoDePerfil: string? – pode ser string ou null.
- id: string – uuid().

☐ Post:

- id: string – uuid().
- userId: string – id de quem postou.
- data: Date – now().



- descricao: string? – pode ser string ou null.
- fotoLink: string – link para foto do post.

7) Busque estudar o que deve ser feito.

O que é necessário ver na [Rocketseat](#):

Formação Node:

- a) Módulo 1: Ver todos os vídeos.
- b) Módulo 2: Ver todos os vídeos.
- c) Módulo 3: Ver todos os vídeos.

Entender o básico é importante. Embora Prisma e Fastify sejam o centro desse projeto piloto, um bom conhecimento de todo o processo vai facilitar o seu trabalho. Use e abuse das capacitações.

Embora possam ter o mesmo resultado final, dominar um projeto e executar um projeto são coisas diferentes.